National University of Singapore
School of Computing
CS1101S: Programming Methodology
Semester I, 2026/2027

R1
Elements of Programming

Review

1. The Source Playground provides a space for writing and testing Source programs, with an *editor* for writing your code.
2. Python §1 is the first in a series of four languages that we will use in this module. Look up the language description of Python §1, provided in the module website: `https://docs.sourceacademy.org/python/python_1/`.
3. Python §1 allows for declaring names. Example from the lecture:

   ```
   size = 2

   5 * size
   ```

   This program consists of two statements. What kind of statements are they?
4. Python §1 allows for `True` and `False` as primitive expressions. These expressions evaluate to *boolean values* that can be used to take yes/no decisions.
5. Python §1 provides for the following comparison operators for numbers: `>`, `<`, `>=`, `<=`, `==` and `!=`. They work as expected: You can place them between two expressions. When such a comparison is evaluated, the two expressions are evaluated first. When they evaluate to numbers, the comparison evaluates to either `True` or `False`, depending on the numbers and the operator. For example, executing the program

   ```
   1 + 2 < 5
   ```

   results in `True`.
6. Function declarations of the form

   ```
   def name(parameters):
       statement
   ```

   declare the given *name* and make it refer to a function with the given parameters and body statement. *Parameters* is a comma-separated sequence of names of variables. For now, the body statement is always a single `return` statement.

   When the function is applied, the return expression is evaluated.

   Example from lecture:

```
def square(x):
    return x * x
```

7. Expressions of the form

   consequent `if` *predicate* `else` *alternative*

   are called *conditional expressions*. First, the *predicate* expression is evaluated. If the result is `True`, the *consequent* expression is evaluated, and if the result is `False`, the *alternative* expression is evaluated.

   Example from the lecture:

```
def abs(x):
    return x if x >= 0 else -x
```

Evaluation of Expressions

Textbook section 1.1.3 explains the evaluation of expressions with a diagram. Read the section before class if you get a chance. We will review the diagram in Figure 1.1 during the reflection session.

Problems:

1. Declare `x` to refer to the value 3, and then evaluate the following statements. Observe the results!

```
print(1 + 1 if True else 17)

print(x if x > 0 else -x)

print(1 if x == 0 else 2)

print(1 if True else (4711 if y < 1 else 42))
```

2. Evaluate the following statements in the Playground:

```
def square(x):
    return x * x

print(square(5))

def hypothenuse(a, b):
    return math_sqrt(square(a) + square(b))

print(hypothenuse(3, 4))
```

3. What do you think happens when the following program is evaluated?

```
1 + 2 * 3 + 4
```

Can you illustrate the computational process with a diagram similar to Figure 1.1? Run the program using the stepper. What do you observe?

4. What do you think happens when the following program is evaluated?

```
4 * 5 if 1 + 2 >= 3 else 6 + 7
```

Can you illustrate the computational process with a diagram similar to Figure 1.1? Run the program using the stepper. What do you observe?